輕量級高維度影像雙邊濾波之

多版本平行化計算與台灣杉三號算力效益評估

期末報告

目錄

1. 概述 3
    1.1 與初始設計之差異 3
2. 問題定義與演算法 4
3. 系統設計與平行化版本 4
    3.1 Serial 4
    3.2 OpenMP 4
    3.3 MPI Static 4
    3.4 MPI Dynamic 5
    3.5 CUDA 5
    3.6 品質評估指標 5
4. 實驗情境與環境設計 5
5. 效能量測指標 5
6. 實驗結果 6
    6.1 時間結果（序列 / OpenMP / MPI） 6
    6.2 加速比與平行效率 7
    6.3 CUDA 結果 8
    6.4 品質結果 8
    6.5 跨架構效能對比（以節點為公平基準） 8
    6.6 MPI 通訊成本拆解（static vs dynamic） 9
    6.7 CPU 純量擴展之核心時間成本（core-seconds） 10
7. 討論與分析 11
    7.1 OpenMP 11
    7.2 MPI static 11
    7.3 MPI dynamic 12
    7.4 CUDA 12
    7.5 跨架構效能與算力成本效益 12
    7.6 輸出品質 13
8. 結論 15
9. 檔案清單 15
10. AI 工具 16

1. 概述

這次的報告中會對灰階影像之雙邊濾波演算法，在**台灣杉三號**與 **Google colab** 上實作，然後比較不同的平行化 Code 的效能與輸出品質。比較對象包含五個不同的版本：Serial、OpenMP、MPI _Static、MPI _Dynamic 和 CUDA 。測試影像是一張 4096×4096 的灰階影像 dog_gray.jpg，所有品質比較都是以 Serial 版輸出 out_serial.jpg 為基準。



dog_gray.jpg

報告中會同時比較「效能」與「算力經濟學成本效益」兩個面向。效能這邊涵蓋執行時間、加速比跟平行效率；然後針對 CPU 與 GPU 的跨架構對比，以「每占用一個實體節點所換得的產出」作為不同硬體間的唯一可比對的公平基準。成本效益面則以「核心數 × 執行時間」為成本模型，量化各配置「多開核心是否划算」的資源效率——因為這次的實驗都是在單一個節點內執行，核心時間才是真正有鑑別力的成本指標。全文透過三張關鍵圖表進行深入分析：強擴展性折線圖（圖一）、跨架構效能對比長條圖（圖二），以及核心時間成本折線圖（圖三）。

1.1 與初始設計之差異

實作階段相較於初始設計做了兩項調整：

(1) 濾波核半徑由 R=7（15×15 視窗）調整為 R=5（11×11 視窗），以在約 3 秒內完成單組實驗、縮短在 Slurm 佇列的排隊時間。

(2) 將原規劃的「MPI 多機」版本改為「MPI _Dynamic master-worker」版本，以比較同節點下 Static 與 Dynamic 的排程差異。

2. 問題定義與演算法

雙邊濾波的核心，是對影像中的每個像素 x，在它周圍 (2R+1)² 的視窗 Ω 內進行加權平均，同時考慮「空間鄰近度」與「色彩（亮度）相似度」兩種權重：

$$I'(x) = \frac{1}{W_x} \sum_{\xi \in \Omega} I(\xi) \exp\left(-\frac{\|x-\xi\|^2}{2\sigma_d^2}\right) \exp\left(-\frac{(I(x)-I(\xi))^2}{2\sigma_r^2}\right)$$

$$W_x = \sum_{\xi \in \Omega} \exp\left(-\frac{\|x-\xi\|^2}{2\sigma_d^2}\right) \exp\left(-\frac{(I(x)-I(\xi))^2}{2\sigma_r^2}\right)$$

然後 σ_d 控制空間域權重的衰減速度、σ_r 控制值域（亮度差）權重的衰減速度，W_x 是正規化常數。本報告採用之參數為：N=4096、RADIUS=5、SIGMA_D=3.0、SIGMA_R=20.0，濾波視窗為 11×11。

3. 系統設計與平行化版本

3.1 Serial

讀入 dog_gray.jpg，一個一個像素套用雙邊濾波然後輸出 out_serial.jpg。同時作為所有品質比較的基準參考。

3.2 OpenMP

在最外層的 row 迴圈加上 #pragma omp parallel for schedule(static)，交由編譯器自行切分列數，程式碼改動最小。輸出 out_openmp.jpg。

3.3 MPI Static

採用 1D Row Decomposition，將影像列數平均分配給各行程。計算前以 MPI_Sendrecv 交換上下各 R 列的鬼影行，計算完成後以 MPI_Gatherv 將各區塊收回 Rank 0 組成完整影像。輸出 out_mpi_static.jpg。

3.4 MPI Dynamic

用 master-worker 設計，並以 chunk 為單位進行動態派工。Rank 0 擔任 master 負責派工與彙整結果，各 worker 主動向 master 請求下一個區塊。這樣的設計可處理負載不均，但會引入區塊請求的往返通訊與排程 cost。輸出 out_mpi_dynamic.jpg。

3.5 CUDA

不做複雜的 Shared Memory，直接以 Global Memory 撰寫。定義二維 Block（測試 4×4、16×16、32×32 三種尺寸），每個 thread 按照它的 threadIdx 與 blockIdx 對應到一個像素 (x, y)，直接讀取 Global Memory 進行濾波運算。量測 H2D 傳輸、Kernel 執行、D2H 傳輸與總時間。輸出 out_cuda.jpg。

3.6 品質評估指標

以 MSE、PSNR、MAE 衡量輸出品質，並在可得時統計差異像素數（Diff px）。所有指標皆以 out_serial.jpg 為基準進行比較。

4. 實驗情境與環境設計

運算平台：台灣杉三號 以及 google colab。

工作負載：固定影像尺寸 4096×4096，固定濾波半徑 R=5。為控制佇列排隊時間，嚴格限制資料規模與核心數。

實驗組合與資源配置如下表：

實驗版本	節點與核心配置	實驗目的
OpenMP	1 節點：1, 2, 4, 8, 12, 16 緒	觀察單節點內共享記憶體的擴展性
MPI static	1 節點：1, 2, 4, 8, 12, 16 行程	同節點下 MPI 記憶體複製 vs OpenMP 緒共享之開銷
MPI dynamic	1 節點：2, 4, 8, 12, 16 行程	主從動態排程於均勻負載下之行為
CUDA	Google colab T4 GPU：block 4×4, 16×16, 32×32	建立異質運算基準線

5. 效能量測指標

- **執行時間 T**：CPU 端以 omp_get_wtime() 或 MPI_Wtime() 包覆純計算與通訊迴圈；CUDA 端 T = T(H2D) + T(Kernel) + T(D2H)。
- **加速比 S_p = T(1) / T(p)**：本報告以序列版時間 T_serial = 5.8295 s 作為跨範式比較的共同基準。
- **平行效率 $E_p = S_p / p \times 100\%$**：為衡量「各範式自身的擴展品質」，效率以各範式自身的單行程 / 單緒時間為基準計算，以避免採用跨範式基準造成的失真（例如 MPI 單行程版本本身即快於序列版）。
- **算力成本效益**：以「核心數 × 執行時間」為成本單位，衡量各配置「多開核心是否划算」；由於本實驗皆於單一節點內執行（節點數恆為 1），核心時間才是真正有鑑別力的算力成本指標。（另對 CPU 與 GPU 的跨架構對比，因核心無法直接相比，改以「每占用一台節點的產出」= 1 ÷（節點數 × 執行時間）作為公平效能基準，詳見 6.5 節。）

6. 實驗結果

6.1 時間結果（序列 / OpenMP / MPI）

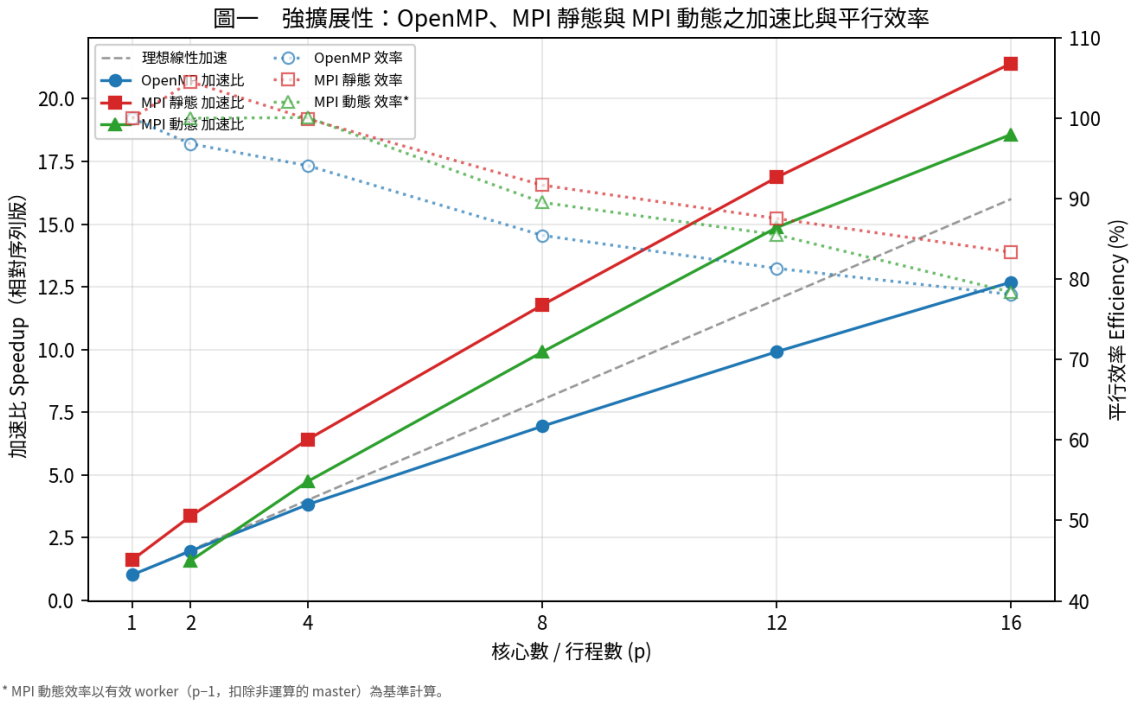
版本	緒數 / 行程數	時間 (s)	加速比
serial	1	5.8295	1.00×
OpenMP	1	5.7397	1.02×
OpenMP	2	2.9642	1.97×
OpenMP	4	1.5249	3.82×
OpenMP	8	0.8398	6.94×
OpenMP	12	0.5881	9.91×
OpenMP	16	0.4594	12.69×
MPI static	1	3.6312	1.61×
MPI static	2	1.7369	3.36×
MPI static	4	0.9087	6.42×
MPI static	8	0.4951	11.77×
MPI static	12	0.3457	16.86×
MPI static	16	0.2723	21.41×
MPI dynamic	2	3.6892	1.58×

版本	緒數 / 行程數	時間 (s)	加速比
MPI dynamic	4	1.2288	4.74×
MPI dynamic	8	0.5887	9.90×
MPI dynamic	12	0.3922	14.86×
MPI dynamic	16	0.3138	18.58×

6.2 加速比與平行效率

下表整理 OpenMP 與 MPI 靜態兩範式的加速比（相對序列版）與平行效率（相對各自單行程）；MPI 動態的加速比則見 6.1 節。圖一將三種 CPU 範式繪於同一張圖上以利對照。須註明：MPI 動態因無單行程資料、且 Rank 0 為非運算的 master，其效率改以「有效 worker（p−1）」為基準計算（以 2 行程 = 1 個 worker 為基準），故與另兩者並非完全對等，僅供趨勢參考。

p	OpenMP 時間(s)	OpenMP 加速比	OpenMP 效率	MPI 時間(s)	MPI 加速比	MPI 效率
1	5.7397	1.02×	100.0%	3.6312	1.61×	100.0%
2	2.9642	1.97×	96.8%	1.7369	3.36×	104.5%
4	1.5249	3.82×	94.1%	0.9087	6.42×	99.9%
8	0.8398	6.94×	85.4%	0.4951	11.77×	91.7%
12	0.5881	9.91×	81.3%	0.3457	16.86×	87.5%
16	0.4594	12.69×	78.1%	0.2723	21.41×	83.3%



*圖一　強擴展性：OpenMP、MPI 靜態與 MPI 動態之加速比與平行效率（ * 動態效率以有效 worker p−1 為基準 ）*

6.3 CUDA 結果

Block 尺寸	H2D (ms)	D2H (ms)	Kernel (ms)	總時間 (ms)
4 × 4	15.26	43.09	59.54	117.88
16 × 16	14.84	14.46	25.11	54.40
32 × 32	14.50	14.09	25.23	53.81

6.4 品質結果

三種 CPU 平行版本與 CUDA 版本相對序列基準之品質指標如下：

版本	MSE	PSNR (dB)	MAE	差異像素 (/16,777,216)
OpenMP	0.136825	56.7691	0.298197	2,357,247
MPI static	0.136825	56.7691	0.298197	2,357,349
MPI dynamic	0.136825	56.7691	0.298197	2,357,349
CUDA	0.000157	86.1620	0.000157	—

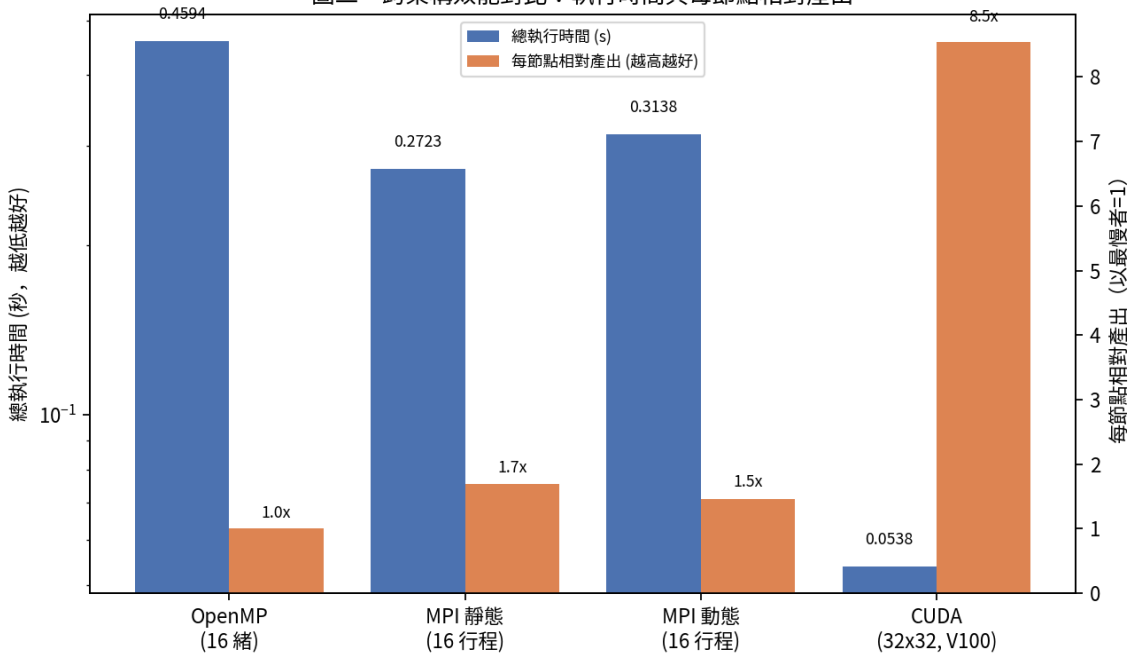
6.5 跨架構效能對比（以節點為公平基準）

取各版本最佳配置進行跨架構對比。由於 GPU 與 CPU 的核心無法直接相比，所以這裡以

「每占用一個實體節點所換得的產出」作為異質硬體間唯一公平的對等基準，其值為 1 ÷（節點數 × 執行時間），並以最慢者正規化為 1。須留意：因四種配置皆使用單一節點（節點數 = 1），此指標在此本質上等同於比較執行時間，唯有在比較「節點數不同」的配置（例如單機 vs 多機 MPI）時才會展現獨立於時間的意義。在此基準下，CUDA 以約 8.5 倍的相對產出勝出，代表每占用一台機器，GPU 的產出最高。至於 CPU 各配置之間真正有鑑別力的算力成本，則於 6.7 節以「核心時間」衡量。

配置	節點數	執行時間 (s)	資源量 (節點·s)	每節點相對產出
OpenMP（16 緒）	1	0.4594	0.4594	1.00×
MPI static（16 行程）	1	0.2723	0.2723	1.69×
MPI dynamic（16 行程）	1	0.3138	0.3138	1.46×
CUDA（32×32, V100）	1	0.05381	0.05381	8.54×

圖二　跨架構效能對比：執行時間與每節點相對產出



圖二　跨架構效能對比：執行時間與每節點相對產出（時間軸為對數刻度）

6.6 MPI 通訊成本拆解（static vs dynamic）

下表進一步拆解 MPI 兩版本的時間組成。可清楚看出 static 版的鬼影行交換成本極低（即使 16 行程也僅約 1.2 ms，佔總時間不到 0.5%），而 dynamic 版因 master-worker 的區塊請求往返，通訊成本高出約一個數量級（約 8–32 ms）。（總 wall time 時間略大於「計算 +

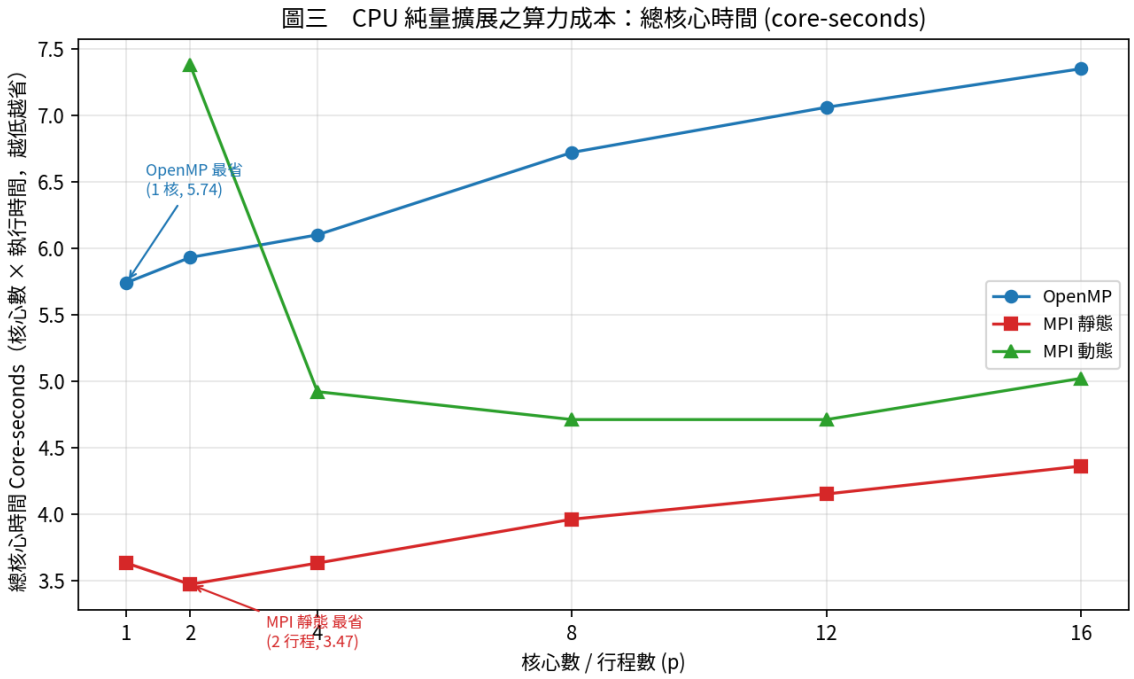
通訊」之和，差額來自影像讀寫等 I/O。）

版本	行程數	計算時間 (s)	通訊時間 (ms)	總牆鐘 (s)	通訊佔比
MPI static	1	3.6066	0.000	3.6312	≈0%
MPI static	2	1.7238	0.311	1.7369	0.02%
MPI static	4	0.9029	0.286	0.9087	0.03%
MPI static	8	0.4915	0.524	0.4951	0.11%
MPI static	12	0.3425	1.420	0.3457	0.41%
MPI static	16	0.2692	1.183	0.2723	0.43%
MPI dynamic	2	3.6573	31.967	3.6892	0.87%
MPI dynamic	4	1.2112	17.606	1.2288	1.43%
MPI dynamic	8	0.5788	9.898	0.5887	1.68%
MPI dynamic	12	0.3833	8.831	0.3922	2.25%
MPI dynamic	16	0.3060	7.780	0.3138	2.48%

6.7 CPU 純量擴展之核心時間成本（core-seconds）

當節點固定為 1 時，CPU 各配置間真正有鑑別力的成本指標是「總核心時間」= 核心數 × 執行時間。它回答的問題是：多開核心換取的加速，是否值得多耗用的核心資源？下表與圖三揭示一個重要且常被忽略的取捨——跑最快的配置，並非最省資源的配置。

p	OpenMP (s)	OpenMP core·s	MPI 靜態 (s)	MPI 靜態 core·s	MPI 動態 (s)	MPI 動態 core·s
1	5.7397	5.74	3.6312	3.63	—	—
2	2.9642	5.93	1.7369	3.47	3.6892	7.38
4	1.5249	6.10	0.9087	3.63	1.2288	4.92
8	0.8398	6.72	0.4951	3.96	0.5887	4.71
12	0.5881	7.06	0.3457	4.15	0.3922	4.71
16	0.4594	7.35	0.2723	4.36	0.3138	5.02



圖三　CPU 純量擴展之算力成本：總核心時間（core-seconds，越低越省）

OpenMP 的總核心時間隨核心數單調上升（5.74 → 7.35 core·s）：平行效率不到 100%，因此每多開一顆核心，因效率遞減而付出的「額外核心時間」持續累積。MPI static 則在 2 行程出現最低點 3.47 core·s（受超線性快取效應之賜），之後才上升至 16 行程的 4.36 core·s。MPI dynamic 因 Rank 0 為純 master 不參與運算，等於恆定浪費一顆核心，其核心時間明顯高於靜態（16 行程 5.02 vs 4.36 core·s）。

結論：若追求最短 wall time，應選 16 核；若追求最省算力，OpenMP 宜採低核心數、MPI static 的甜蜜點則落在約 2–4 行程。此「最快 ≠ 最省」的取捨，正是節點數固定時，成本效益分析仍能提供的關鍵洞見。

7. 討論與分析

7.1 OpenMP

OpenMP 隨 thread 數提升展現良好擴展性，16 緒時取得 0.4594 s 的最佳時間。平行效率自 100% 緩降至 78.1%，下滑主因為高緒數下的記憶體頻寬競爭，以及最外層列切分在快取層級的局部性效應。整體而言改動成本最低、易於維護。

7.2 MPI static

實驗結果顯示為 CPU 端最佳，16 行程時達 0.2723 s（相對序列 21.41×）。值得注意的是其

在 p=2 出現略為超線性（效率達 104.5%）的現象：橫條切分後各行程的工作集較小，更容易放入私有快取（L2/L3），其帶來的效益足以抵銷少量的鬼影行交換成本；且在單一節點上，鬼影行交換走的是節點內記憶體複製，成本極低。實測通訊成本拆解（6.6 節）即印證此點——即使在 16 行程下，通訊時間也僅約 1.2 ms、佔總時間不到 0.5%，因此幾乎不構成擴展瓶頸。

7.3 MPI dynamic

由實測通訊時間可見，dynamic 版的通訊開銷（約 8–32 ms）較 static 版（< 1.5 ms）高出約一個數量級，這源自 worker 反覆向 master 請求區塊的往返成本。更關鍵的是，由於 Rank 0 為純 master 不參與運算，動態版在 p 個行程下實際只有 p−1 個 worker；最直接的證據是動態版 2 行程（僅 1 個 worker）的 3.6892 s 幾乎等同於靜態版 1 行程的 3.6312 s。因此在均勻負載下，dynamic 版相當於少用一顆核心，整體略慢於 static（16 行程 0.3138 s vs 0.2723 s）。但其擴展性仍佳；在「負載不均」情境下（本實驗為均勻負載故無法展現此優勢），動態派工能避免靜態切分常見的尾端拖延（straggler）問題。

7.4 CUDA

32×32 與 16×16 配置最快，其中 32×32 以 53.81 ms 的總時間為全場最快。4×4 因每個 block 僅 16 個 thread，佔用率（occupancy）過低且無法填滿一個 warp（32 thread），導致 kernel 與傳輸時間皆顯著較高（117.88 ms）。另須注意：CUDA 總時間中資料傳輸（H2D + D2H ≈ 28.6 ms）佔比超過一半，而 kernel 僅 25.23 ms；對單張小影像而言，PCIe 傳輸已成為瓶頸，若改以批次處理多張影像即可有效攤平此傳輸成本。

7.5 跨架構效能與算力成本效益

跨架構效能（以節點為公平基準）：由於四種配置皆使用單一節點，「節點 × 時間」在此本質上等同於比較速度；其價值在於提供 CPU 與 GPU 異質硬體間唯一可對等的基準——每占用一台機器所換得的產出。在此基準下，CUDA 以約 8.5 倍的相對產出勝出，凸顯 GPU 在像素獨立、高度資料平行之影像處理上的絕對優勢。須補充：此基準僅計入「節點時數」，未計入 GPU 節點較高的硬體單價；若改用價格加權模型，GPU 的領先幅度會縮小，但在本工作負載下仍具優勢。

CPU 純量擴展（以核心時間為單位）：當節點固定為 1 時，CPU 各配置間有鑑別力的成本指

標是核心時間（見 6.7 節）。分析顯示「最快的配置並非最省資源的配置」——OpenMP 的核心時間隨核心數遞增、MPI static 則在約 2–4 行程達到最低，代表追求極速（16 核）與追求資源效率（少核心）之間存在實質取捨。此外，dynamic 版因 master 核心閒置，核心時間始終高於靜態，從算力經濟的角度進一步說明了動態派工在均勻負載下的劣勢。

7.6 輸出品質

三種 CPU 平行版本（OpenMP / MPI static / MPI dynamic）的 MSE / PSNR / MAE 完全一致（PSNR 56.77 dB），其差異像素僅源自浮點累加順序不同造成的末位捨入，屬於數值上無害的差異。CUDA 版 PSNR 高達 86.16 dB、MSE 僅 0.000157，與序列版幾乎完全一致，顯示 GPU 浮點運算在此精度需求下亦高度可靠。



out_mpi_dynamic_P16



out_mpi_static_P16



out_openmp_T16



out_cuda

8. 結論

雙邊濾波在平行化下獲得顯著加速。在 CPU 版本中，MPI static 切分於本資料集提供最佳整體執行時間（0.2723 s）；CUDA 則以 53.81 ms 取得全場最快的 wall time，且輸出品質最接近序列基準。

綜合效能與每節點產出來看，於像素獨立的影像濾波任務上，GPU 為最佳選擇；而在純 CPU 場景中，單節點內 MPI static 優於 OpenMP，MPI dynamic 則因在均勻負載下的排程開銷而略遜一籌。此外，本實驗亦驗證了一項重要觀念：MPI 通訊成本的高低高度取決於部署拓樸——單節點內的鬼影行交換成本極低，唯有跨節點時網路延遲才會成為主導因素。

最後，從算力經濟學角度，本報告指出「最快的配置未必最省資源」：在單節點限制下以核心時間衡量，OpenMP 與 MPI static 的最省資源點都落在較低核心數，與追求最短 wall time 的 16 核配置形成明確取捨。此一觀點提醒：平行化的目標究竟是「縮短時間」還是「節省算力」，會導向不同的最佳配置選擇。

9. 檔案清單

- bilateral_serial.c
- bilateral_openmp.c
- bilateral_mpi.c
- bilateral_mpi_dynamic.c
- bilateral_cuda.ipynb
- compare_quality.c
- results_serial.csv
- results_openmp.csv
- results_mpi_single.csv
- results_cuda.csv
- quality_openmp_T{1,2,4,8,12,16}.txt
- quality_mpi_static_P{1,2,4,8,12,16}.txt
- quality_mpi_dynamic_P{2,4,8,12,16}.txt
- quality_cuda.txt

10. AI 工具

Claude code:

使用 AI tools 協助生成圖檔

使用 AI tools coding 以及分析目前實驗的情況

Cladue:

與 AI 討論實驗數據以及數據代表的涵義

請 AI 分析段落是否敘述正確以及改寫

使用 AI tools 協助版面調整

Codex:

檢查我的程式語法是否正確